AlertaBairro

Monitoramento colaborativo de ocorrências por chuva

Projeto acadêmico CitizenTech | São Paulo (SP)

Identificação: Nome: _____ RM: _____ Turma: _____

PONTO DE PARTIDA

Problema e cidadão afetado

Em dias de chuva forte, moradores e pessoas em deslocamento na cidade de São Paulo podem encontrar vias alagadas, bueiros obstruídos e trajetos prejudicados. É difícil reunir em um lugar informações comunitárias simples sobre esses transtornos. O AlertaBairro propõe um painel de **relatos colaborativos não verificados** combinado com a tendência meteorológica para a cidade.

EVIDÊNCIA PÚBLICA • CEMADEN

2.505
alertas no Brasil

49
São Paulo

alertas geo-hidrológicos em 2025
no Brasil; 49 para São Paulo.

Solução proposta

O cidadão escolhe apenas uma **região ampla**, um **tipo de ocorrência** e sua **intensidade**. A API Flask grava o relato no MySQL e apresenta registros recentes. Um painel complementar consulta dados horários públicos da Open-Meteo, sem transformar a previsão em indicador de risco. A moderação pode atualizar o estado dos registros e excluí-los.

CONEXÃO COM OS OBJETIVOS DE DESENVOLVIMENTO SUSTENTÁVEL

ODS 11

Cidades sustentáveis

Informação comunitária para mobilidade e resiliência urbana.

ODS 13

Ação climática

Acesso à informação de chuva para maior atenção aos impactos.

ODS 9

Infraestrutura

Aplicação de API, dados e serviços digitais para cidades.

Limite ético: não há previsão de enchente, validação oficial, geolocalização exata ou emissão de alertas de emergência. Os dados do protótipo devem ser fictícios ou abertos.

ARQUITETURA E IMPLEMENTAÇÃO

Como a solução funciona

Módulos Python separados, API REST e SQL sem ORM

FLUXO DE COMPONENTES

NAVEGADOR
HTML / CSS / JavaScript

API FLASK
Rotas e validações

MYSQL 8.4
CRUD com PyMySQL

OPEN-METEO
requests + JSON público

LOG DE AÇÕES
Arquivo, data e resultado

SOCKET TCP LOCAL
Comando STATUS / JSON

Modelo de dados: tabela ocorrencias

Campo	Tipo / regra	Finalidade
id	INT UNSIGNED, PK	Identificador automático
regiao	VARCHAR(20), lista fixa	Centro, Norte, Sul, Leste, Oeste
tipo	VARCHAR(24), lista fixa	Alagamento, bueiro, via
intensidade	VARCHAR(10), lista fixa	Baixa, moderada, alta
status	VARCHAR(12), lista fixa	Aberto, em análise, resolvido
origem	VARCHAR(12)	Demonstração ou comunidade
criado_em / atualizado_em	TIMESTAMP (UTC)	Histórico de criação e alteração

Endpoints e status HTTP

GET	/api/ocorrencias	200; 422 / 503
POST	/api/ocorrencias	201; 400 / 415 / 422 / 429
PUT	/api/ocorrencias/{id}	200; 403 / 404 / 422
DELETE	/api/ocorrencias/{id}	204; 403 / 404
GET	/api/clima · /api/resumo	200; 503

Organização: app/__init__.py (Flask e logs); db.py (SQL parametrizado); validacao.py (listas de valores); clima.py (requests); limite.py (controle de cadastros); tcp_server.py (rede local).

SEGURANÇA, TESTES E VIABILIDADE

Avaliação responsável

Boas práticas alinhadas aos requisitos de Coding for Security

TRÊS AMEAÇAS E CONTROLES

01 Injeção SQL

Consultas com placeholders %s e parâmetros separados no PyMySQL; valores aceitos apenas por listas predefinidas.

02 Alteração ou exclusão indevida

PUT e DELETE exigem chave administrativa longa no header; comparação por hmac.compare_digest; chave em .env.

03 Abuso e relatos falsos

Limite simples de até cinco POST/minuto por origem, JSON limitado a 8 KB e ausência de campos livres/pessoais.

Privacidade e LGPD

Os relatos armazenam **somente região ampla, categoria, intensidade, situação, origem e datas**; não solicitam nome, telefone, e-mail ou endereço exato. Senhas ficam no .env (ignorado no Git), os logs não guardam tokens ou corpo das requisições e erros retornam mensagens genéricas. O controle de taxa mantém endereços IP apenas **temporariamente em memória**, sem registrá-los em arquivo. A publicação pública exigiria avaliação adicional de proteção de dados.

Validação do protótipo

Foram preparados testes automatizados com **pytest** para códigos HTTP, entradas inválidas, autorização, erros genéricos e resposta simulada da Open-Meteo. A análise estática de sintaxe Python e JavaScript pode ser executada separadamente. **Execução real integrada** com MySQL, Flask, navegador e Open-Meteo deve ser feita pelo estudante em ambiente com dependências e Docker instalados; o resultado não é presumido neste relatório.

Limitações e continuidade

A previsão usa coordenadas centrais de São Paulo, não capta microclimas; relatos não passam por verificação; o limite em memória reinicia com o aplicativo. Para produção, seriam necessários autenticação individual, HTTPS, moderação, monitoramento, acessibilidade e integração com fontes oficiais. O TCP é restrito a 127.0.0.1 e retorna contagens agregadas.

Recursos extras: Docker Compose facilita reproduzir o MySQL; pytest e GitHub Actions automatizam verificações; cabeçalhos de segurança e rate limiting reforçam a defesa. Esses recursos são complementares, não indispensáveis ao CRUD.

ÉTICA, AUTORIA E FONTES

Uso de IA: planejamento, implementação e documentação foram elaborados com assistência de ChatGPT (OpenAI). A avaliação é individual: o estudante deve revisar, compreender e demonstrar o código. Não há publicação de repositório GitHub por este documento.

[1] FIAP Experience: experiência CitizenTech

[2] Cemaden: Relatório Estado do Clima 2025, p. 32

[3] Open-Meteo: documentação da Forecast API

[4] ONU Brasil: ODS 11